Banking System

Sprint 1: System Design

Table of Contents

Table of Contents **2**
System Interaction with the Environment **2**
CRC Cards **2**
System Architecture **2**
System Decomposition **2**

System Interaction with the Environment

Dependencies

1. NodeJS is used to handle server-side programming and dependency importing.
2. Angular framework is used to generate the frontend view for customers.
3. Java is used for backend logic to manage the database.
4. MySQL database is used to model and store data.

Assumptions

1. The system will be deployed and accessed over a modern web browser.
2. An internet connection is necessary for functionality of the system.

CRC Cards

<table>
<tr><td colspan="2">Class:AccountController</td></tr>
<tr><td colspan="2">Parent Class: N/A
Subclasses: N/A</td></tr>
<tr><td>Responsibilities:
• Handle CRUD operations
○ Post
○ Get
○ Put
○ Delete</td><td>Collaborators: N/A</td></tr>
</table>

Class:CustomerInfoController	
Parent Class: N/A Subclasses:N/A	
Responsibilities: ● Handle CRUD operations ○ Post ○ Get ○ Put ○ Delete	Collaborators: N/A

Class:AccountDAO	
Parent Class: N/A Subclasses: N/A	
Responsibilities: ● Provide an abstract for expected account operations	Collaborators: ● AccountService

Class:CustomerInfoDAO	
Parent Class: N/A Subclasses: N/A	
Responsibilities: ● Provide an abstract for expected customer operations	Collaborators: ● CustomerInfoService

Class:LocalAccountDataAccessService	
Parent Class: N/A Subclasses: N/A	
Responsibilities: ● Provide an implementation for account operations	Collaborators: AccountDAO

Class:LocalCustomerDataAccessService

Parent Class: N/A Subclasses: N/A	
Responsibilities: ● Provide an implementation for customer operations	Collaborators: CustomerDAO

Class:Account	
Parent Class: N/A Subclasses: N/A	
Responsibilities: ● Models the attributes and methods of accounts ○ ID, balance ● Getters and setters	Collaborators: ● AccountOwners ● CustomerInfo ● Transaction ● AccountService

Class:AccountOwners	
Parent Class: N/A Subclasses: N/A	
Responsibilities: ● Assigns unique owners to their respective account	Collaborators: ● Account

Class:CustomerInfo	
Parent Class: N/A Subclasses: N/A	
Responsibilities: ● Models the attributes and methods of accounts ○ ID, email, password, mobile number, address ● Getters and Setters	Collaborators: ● Account ● SavedPayees ● AccountService

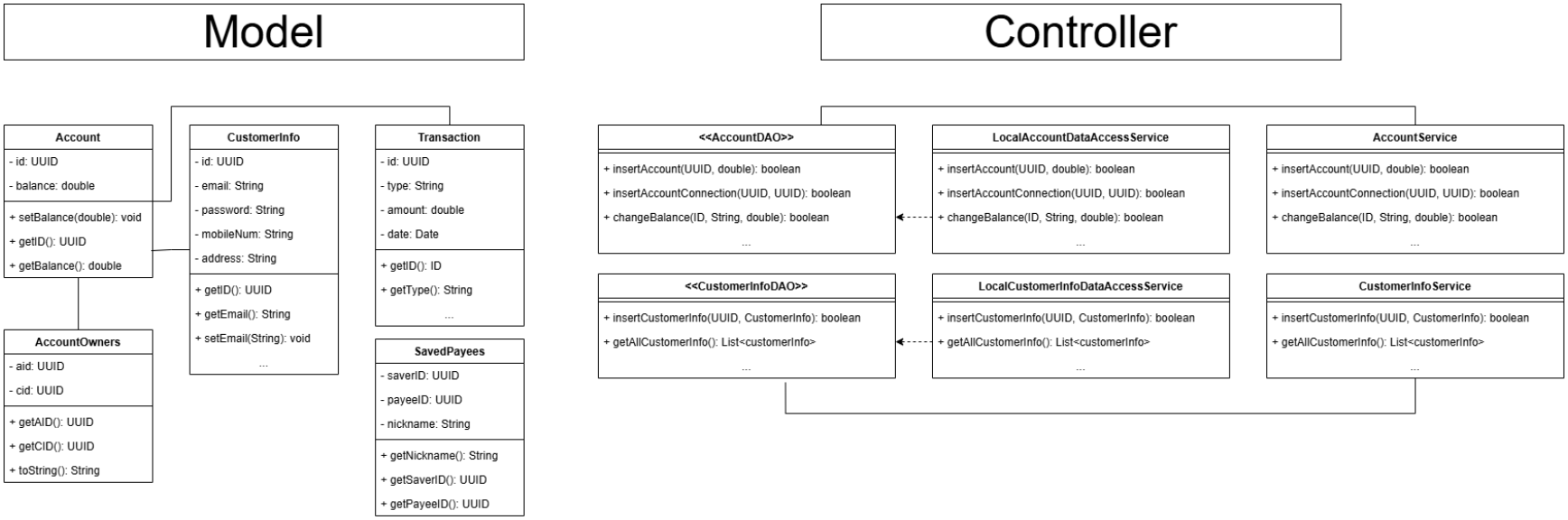
Class:Transaction

Parent Class: N/A Subclasses: N/A	
Responsibilities: ● Models the attributes of a transaction ○ ID, deposit/withdraw, amount, date ● Getters and Setters	Collaborators: ● CustomerInfo ● Account

Class:AccountService	
Parent Class: N/A Subclasses: N/A	
Responsibilities: ● Applies the operations of Accounts while associating other classes ○ Account creation, Balance change ● Getters and Setters	Collaborators: ● AccountDAO ● Account ● CustomerInfo ● Transaction

Class:CustomerInfoService	
Parent Class: N/A Subclasses: N/A	
Responsibilities: ● Applies the operations of CustomerInfo while associating other classes ○ Add new info ● Getters and Setters	Collaborators: ● CustomerDAO

System Architecture



Overview

The system architecture follows the MVC architecture where the classes within the model contain many-to-many relations and the controller contains one-to-one relations.

System Decomposition

Error Handling

- Invalid user input is handles in both the Account and CustomerInfo controllers
    - 400 errors are through if information is invalid or missing